Benchmarking approximate nearest neighbour search: HNSW, IVF-Flat and IVF-PQ

CS4.406 Assignment 3

Setup

Machine. 13th Gen Intel(R) Core(TM) i7-13650HX laptop, 10 physical cores (20 threads), 7.6 GiB RAM, WSL2 Linux. Python 3.11.7, FAISS 1.15.1 (AVX2), hnswlib 0.8.0, numpy 2.4.6. Everything ran on this one machine. **Data.** SIFT (1,000,000 × 128) and GloVe-100 (1,183,514 × 100, vectors and queries normalised); Euclidean distance everywhere. The 10,000 queries are split in half: I choose settings on the first 5,000 (tune) and report on the last 5,000. **Measuring.** Recall@10 uses my own function: an answer counts as correct if its true distance is no larger than the 10th true distance, so ties are not mistakes (unit tests in `tests/`). Speed is queries per second (QPS), one query at a time on one thread, median of 3 timings (15 in Part 4). **Settings tried.** HNSW: M 8, 16, 32 (efConstruction 200). IVF-Flat: nlist 256, 1024, 4096. IVF-PQ: nlist 1024 with 8-bit codes and four values of m (16 to 128 on SIFT, 10 to 100 on GloVe), plus nlist 4096. For each, efSearch or nprobe is swept up to recall 0.99 or until recall stops rising. Every run is a row of `results/results.csv`; `bash run_all.sh` regenerates all tables and figures.

Part 1: exact baseline

Exact search is FAISS `IndexFlatL2`. It agrees with the neighbours stored in the data files: recall is 1.0 in both directions on both datasets. The 10 ids are identical for all but 65 SIFT queries and 1 GloVe query, and in those the answers differ only by tied distances, which is why ties have to be handled.

	one query at a time, 1 thread	one batch, 1 thread	one batch, all physical cores
SIFT	25.8 ms (39 QPS)	7.4 ms (136 QPS)	2.8 ms (357 QPS)
GloVe-100	23.1 ms (43 QPS)	6.1 ms (164 QPS)	2.7 ms (374 QPS)

Why one query at a time is so slow. A single query has to read the whole dataset (512 MB for SIFT) from RAM, so it is limited by memory bandwidth: 25.8 ms is about 20 GB/s on one core. In a batch, the distances are computed as a matrix product over blocks of vectors that stay in cache while every query uses them, so each vector is read from RAM once per batch instead of once per query. That is 3.5× (SIFT) and 3.8× (GloVe) faster on one thread, and splitting the batch over the cores adds 2.6× and 2.3×. I inferred the bandwidth explanation from the timings; I did not measure cache misses.

Part 2: the three indexes (questions 1 and 2)

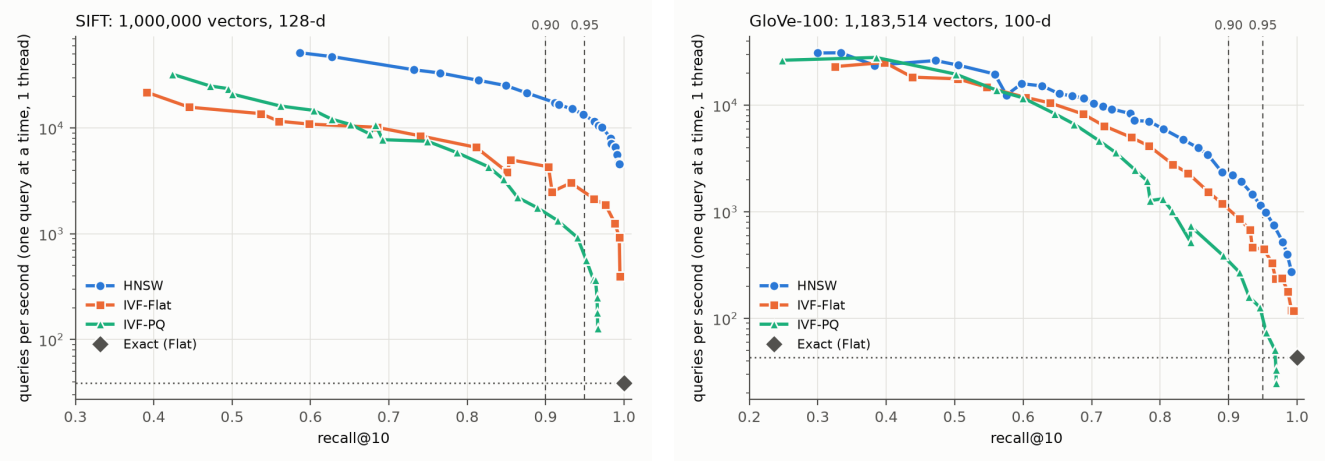


Figure 1. Recall@10 against speed. Each line is the best trade-off curve of an index over all its build settings, chosen on the tune queries and drawn with report-query numbers. Dashed lines mark recall 0.90 and 0.95. The dotted line is exact search.

dataset	index	QPS at recall 0.90	recall reached	QPS at recall 0.95	recall reached	setting at 0.95
SIFT	Exact (Flat)	38.8	1.000	38.8	1.000	-
SIFT	HNSW	16,601	0.917	11,521	0.962	M=32, efSearch=40
SIFT	IVF-Flat	3,031	0.932	2,148	0.962	nlist=4096, nprobe=48
SIFT	IVF-PQ	1,339	0.915	564	0.952	nlist=1024, m=128, nprobe=32
GloVe-100	Exact (Flat)	43.4	1.000	43.4	1.000	-
GloVe-100	HNSW	2,218	0.906	984	0.954	M=32, efSearch=384
GloVe-100	IVF-Flat	861	0.916	445	0.952	nlist=4096, nprobe=384
GloVe-100	IVF-PQ	270	0.916	74.2	0.955	nlist=1024, m=100, nprobe=256

Table 1. QPS at recall 0.90 and 0.95. The setting is the one that was fastest on the tune queries; QPS and recall are the report-query values, so the recall reached can differ a little from the target.

Q1. At recall 0.95, which index is fastest, and does it change between datasets?

HNSW is the fastest on both datasets, so the answer does not change. What changes is the size of the lead. On SIFT HNSW is 5.4× faster than IVF-Flat and 297× faster than exact search; on GloVe it is only 2.2× and 23×. IVF-PQ is last on both, and on GloVe it is just 1.7× faster than exact search. GloVe is harder for every index: it needs efSearch 384 against 40 and nprobe 384 against 48.

Why. HNSW walks a graph. It compares the query with a small set of vectors: at most about efSearch × 2M, a few thousand on SIFT (an upper estimate; I did not count them). IVF must scan every vector in the lists it opens. At recall 0.95 that is 48 of 4,096 lists (about 12,000 vectors) on SIFT, but 384 of 4,096 lists (about 110,000 vectors, 9% of the data) on GloVe, so IVF loses more ground there. IVF-PQ opens the same lists as IVF-Flat, but it computes distances from lookup tables one byte at a time, which seems slower than IVF-Flat's contiguous vector arithmetic (I did not profile this). I think GloVe is harder because its nearest neighbours are barely closer than other points, so each search has to look at far more candidates; I did not measure this (for example with relative contrast).

Q2. What do the indexes cost in build time and memory? What does PQ save and give up?

dataset	index	build setting at 0.95	build time (s)	index size (MiB)	bytes per vector
SIFT	Exact (Flat)	-	0.2	488	512
SIFT	HNSW	M=32, efConstruction=200	86.4	752	788
SIFT	IVF-Flat	nlist=4096	113.3	498	522
SIFT	IVF-PQ	nlist=1024, m=128	73.3	130	137
GloVe-100	Exact (Flat)	-	0.2	451	400
GloVe-100	HNSW	M=32, efConstruction=200	204.4	763	676
GloVe-100	IVF-Flat	nlist=4096	93.5	462	409
GloVe-100	IVF-PQ	nlist=1024, m=100	64.3	122	108

Table 2. Cost of the setting that reaches recall 0.95 (build uses 10 threads; size is the serialised index).

HNSW costs the most memory: 788 (SIFT) and 676 (GloVe) bytes per vector, against 512 and 400 for the raw vectors, because it stores the vectors plus the graph links (2M per point at the bottom layer). It also takes long to build (86 s and 204 s with M=32), since every insertion is a search. **IVF-Flat** adds almost no memory (522 and 409 bytes per vector; the centroids and ids are small). Its build time is k-means, and it grows with nlist: 113 s for nlist 4,096, but only 10 s for nlist 1,024 on SIFT. **PQ** stores each vector as m bytes. It saves 3.8× over IVF-Flat on both datasets (137 against 522 bytes, 108 against 409) and builds in a similar time (73 s and 64 s).

What PQ gives up is accuracy. A distance computed from 8-bit codes has an error, and when it is larger than the gap between the 10th neighbour and the points behind it, true neighbours are ranked wrongly however many lists are scanned. Scanning every list gives the same recall as the sweep (SIFT m=64: 0.872 against 0.881), so it is the codes that limit it. Only m = d (1 dimension per byte) reached 0.95 on both datasets, with a ceiling of about 0.97. It also gives up speed (Table 1).

Part 3: compression on SIFT (plain IVF-PQ against IVF-PQ with exact re-ranking of the top 100)

Hypothesis (committed in `c8c6adc`, before any Part 3 code or result): a smaller m lowers plain recall (ceiling about 0.6 at m=16, 0.97 at m=128); re-ranking helps most when m is small; it costs 10 to 30% of speed at the same nprobe but is faster at equal recall; a smaller m scans faster; re-ranking saves no memory, because the 512-byte vectors must be kept. **Setup.** One IVF-PQ index (nlist 1024) per m = 16, 32, 64, 128. The plain and re-ranked versions (FAISS `IndexRefineFlat`, top 100) search the same codes; nprobe is swept as in Part 2.

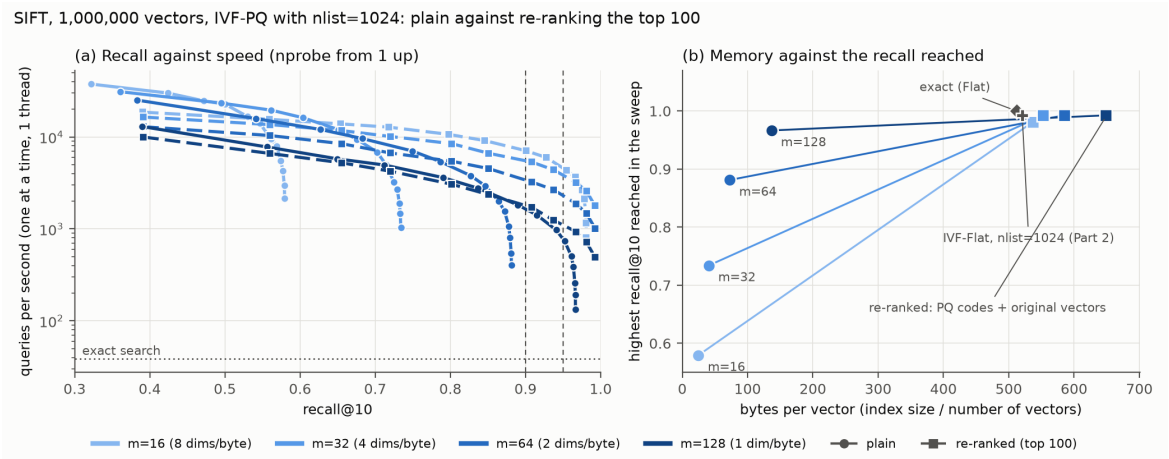


Figure 2. (a) Recall against speed for every m, plain (circles) and re-ranked (squares). (b) Memory against the highest recall reached.

m	version	bytes per vector	highest recall	QPS at recall 0.90	QPS at recall 0.95
16	plain	25	0.580	not reached	not reached
16	re-ranked (top 100)	537	0.981	5,977	3,676
32	plain	41	0.734	not reached	not reached
32	re-ranked (top 100)	553	0.992	4,383	3,201
64	plain	73	0.881	not reached	not reached
64	re-ranked (top 100)	585	0.993	2,662	1,881
128	plain	137	0.967	1,415	739
128	re-ranked (top 100)	649	0.993	1,242	930
-	IVF-Flat, nlist=1024 (Part 2, for reference)	521	0.993	2,003	1,367

Table 3. Last row: Part 2's IVF-Flat with the same nlist, timed in an earlier session. QPS is at the first nprobe that reached the target on the tune queries.

Did the results support the hypothesis? Partly: three of the five predictions held cleanly. *Supported:* plain recall falls with m (highest 0.58, 0.73, 0.88, 0.97), and scanning every list does not help (0.553, 0.711, 0.872, 0.965), so the codes are the limit. Plain speed roughly halves each time m doubles. Re-ranking saves no memory: 25, 41, 73, 137 bytes per vector become 537, 553, 585, 649, more than IVF-Flat's 521. *Right direction, wrong size:* re-ranking helps most at small m, but m=16 reached 0.98, not the 0.85 to 0.9 I expected; even at 8 dimensions per byte the PQ top 100 holds 96% of the true top 10. *Half right:* at the same nprobe, re-ranking cut speed by a median of 12 to 30% (up to 50% at small nprobe). At equal recall it was faster only at 0.95 and m=128 (930 against 739 QPS), and 12% slower at 0.90; for smaller m plain PQ never reaches either target. **Not predicted:** re-ranked m=16 is the fastest IVF at recall 0.95 (3,676 QPS against 1,367 for IVF-Flat) at about the same memory. I think it is because it reads 16 bytes per candidate instead of 512; I did not test that. **Not tested:** other nlist, other re-ranking depths, GloVe, vectors kept on disk.

Part 4: drifting data (my own synthetic clusters)

Hypothesis (committed in `7eb8005`, before any Part 4 code or result): IVF-PQ degrades most, because its centroids and codebooks are learned from old data; IVF-Flat gets slower (a few lists grow fat) but keeps its recall; HNSW is least affected; retraining fixes IVF; rebuilding HNSW is the most expensive recovery; the control (shift 0) does not change.

What I changed. One thing: how far the data that arrives after the build is shifted. The generator (`annbench/drift.py`, seeded) draws 100 Gaussian clusters in 64 dimensions. Each index is built on 200,000 old vectors; 100,000 new vectors are then added without retraining, and 2,000 queries (1,000 to tune, 1,000 to report) are drawn like the new vectors. New vectors and queries are moved along one fixed direction by 0 (the control), 1, 2 or 4 times a typical vector's distance to its cluster centre. Nothing else changes. Each index keeps one search setting, chosen on the control to reach recall 0.95 (efSearch 64, nprobe 6 and 6; IVF-PQ uses m = 64). I measure each **stale** (new vectors just added) and **retrained** (rebuilt on all 300,000).

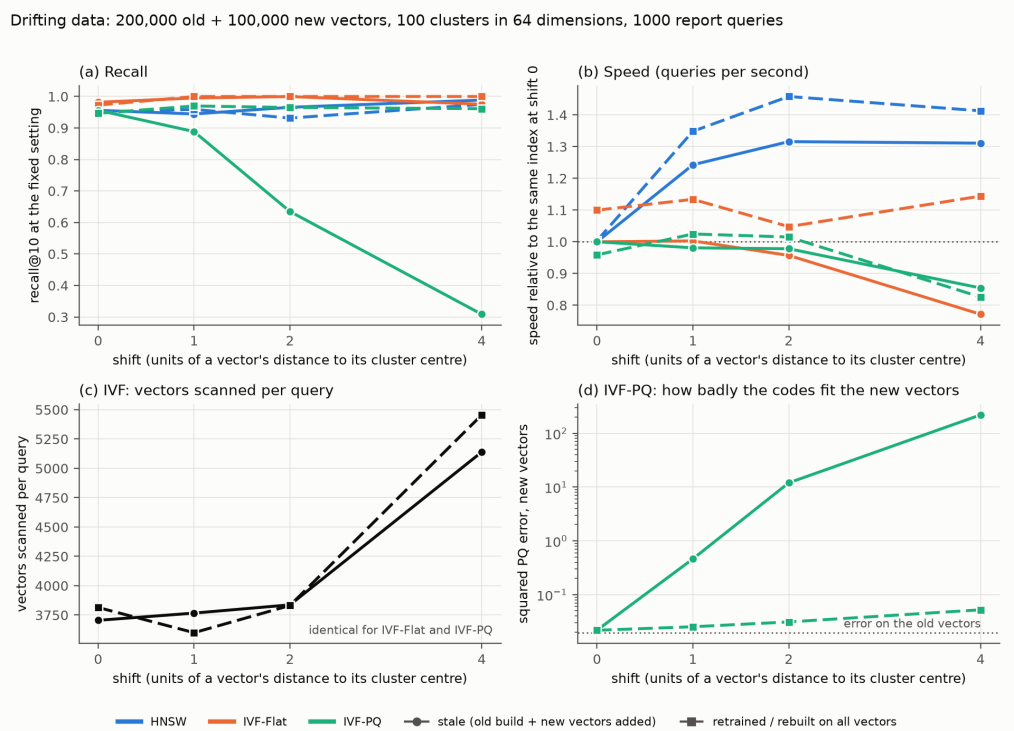


Figure 3. (a) Recall and (b) speed at the fixed setting; (c) vectors scanned per query by the IVF lists (identical for IVF-Flat and IVF-PQ); (d) squared error of the PQ codes on the new vectors. Solid lines are stale indexes, dashed lines retrained ones.

index	measure	shift 0 (control)	shift 1	shift 2	shift 4
HNSW	recall stale	0.956	0.944	0.966	0.989
	recall retrained	0.951	0.959	0.932	0.975
	speed, stale / retrained	1.00	0.92	0.90	0.93
IVF-Flat	recall stale	0.982	0.995	1.000	0.975
	recall retrained	0.973	1.000	1.000	1.000
	speed, stale / retrained	0.91	0.88	0.91	0.67
IVF-PQ	recall stale	0.956	0.889	0.635	0.309
	recall retrained	0.947	0.970	0.965	0.961
	speed, stale / retrained	1.04	0.96	0.96	1.03

Table 4. Recall, and speed of the stale index divided by speed of the retrained one at the same shift.

What happened and why. IVF-PQ collapses: its recall falls from 0.96 to 0.89, 0.64 and 0.31, and retraining brings it back to 0.96 to 0.97. The cause is measured: the codebook was learned on the residuals of the old vectors, so a shifted vector is far from its list centre and its code fits badly. The squared error of the code on the new vectors is 0.022, 0.46, 12 and 220 at shifts 0, 1, 2 and 4 (0.022 to 0.052 after retraining). Wrong distances put the wrong vectors in the top 10, and no search setting can repair that. IVF-Flat keeps its recall until the largest shift (0.975 against 1.000) but is then about 30% slower than the retrained index. HNSW keeps its recall, and the stale graph is about 10% slower. I have no mechanism for HNSW beyond the fact that it has nothing trained in advance, so each new vector links to its nearest neighbours when added.

Supported: IVF-PQ is hit hardest and retraining fixes it; HNSW is least affected; the control stays the same (stale and retrained agree within 0.01 in recall at shift 0). **Not supported:** IVF-Flat did not become "several times slower" (about 1.5 times, at shift 4 only), and my fat-list explanation failed: the stale and retrained indexes have similar largest lists (3,085 against 2,979) and scan similar numbers of vectors (5,139 against 5,456), so I do not know why the stale IVF-Flat is slower. The cost prediction was also wrong at this size: rebuilding HNSW took 9.5 s, not minutes, and IVF-PQ was the most expensive to retrain (22.9 s, for its codebooks).

Not tested, and how far to trust it. Gradual drift, deletions, rotations, new clusters, other dimensions or sizes, real data, other nlist or M, other seeds. HNSW graphs differ between runs: an earlier run of this experiment (3 timing repeats; kept in `results/earlier_run_part4_3repeats.csv`) gave HNSW recalls up to 0.04 apart for the same build, so smaller HNSW differences mean nothing. Speeds are noisy too (up to 28% between my two runs); only the IVF-Flat slowdown at shift 4 and the HNSW gap appeared in both.

Questions 3 to 5

Q3. What did Parts 3 and 4 show, and when could the headline numbers mislead?

Part 3 showed that the headline of each compressed index is only half of the story. PQ's "137 bytes per vector" hides a recall that cannot go above 0.97, and below m = d not above 0.88, whatever search effort is spent. Adding re-ranking restores recall (0.98 to 0.99 for every m) but brings the memory back to 537 to 649 bytes per vector, because the original vectors must stay in RAM. So a memory number needs the recall it was reached at, and a recall number needs the memory that paid for it. **Part 4** showed that every number in Parts 2 and 3 assumes the queries and later vectors look like the data the index was built on. When they do not, a trained index can fail badly (IVF-PQ at 0.31 recall with the search setting that gave 0.95 before) while its speed looks normal, so nothing warns you; HNSW and, until the largest shift, IVF-Flat held up. So the headline numbers could mislead when (1) the data drifts after the build, (2) only one of recall, speed and memory is quoted, (3) speed is single-thread, one query at a time, which says nothing about batched or multi-threaded use (exact search gains 3.5× from batching alone), and (4) the settings were tuned on one query set, which is why I report on a second half.

Q4. 150,000 vectors, or 10 million; 100 ms per query; 16 GB RAM. Would I use ANN, and which index?

I assume SIFT-like vectors (128 dimensions, float32) and recall 0.95 at 10. Exact search reads all the vectors, so its time grows with their number. I did not time other sizes; I scale my 1,000,000-vector timings (25.8 ms per query on SIFT, 23.1 ms on GloVe, one thread).

150,000 vectors: no ANN, use exact search (`IndexFlatL2`). It should take about 3.9 ms per query (SIFT scaling; about 2.9 ms for GloVe's 100 dimensions), 25 times under the budget, with recall 1.0, 77 MB of memory, no build and nothing to tune. An ANN index would only add tuning and a recall below 1. The one reason to use one is throughput: single-thread exact search would manage about 250 queries per second, and HNSW would give about 10,000 or more (Table 1, at 1M).

10 million vectors: use ANN. Exact search would take about 260 ms per query, 2.6 times the budget, and I did not time one query split over several cores (Part 1 timed only batches). Everything fits in 16 GB: raw vectors take 5.1 GB. I would use **HNSW with M=32 and efConstruction 200** (about 7.9 GB at 788 bytes per vector; M=16 would need about 6.6 GB), starting from efSearch 40 on SIFT-like data and about 400 on GloVe-like data, the values that reached 0.95 at 1M. At 1M those cost 0.09 ms and about 1 ms per query, so even a 10 times slower search at 10M is far inside the budget. The real cost is the build: 66 to 86 s per million on 10 threads, so about 11 to 15 minutes at least, and probably more because insertion gets slower as the graph grows. If the build time or the memory of the graph is a problem, IVF-Flat (nlist about 16,384, about 5.2 GB) is the simpler second choice: at 1M it needed nprobe 48 and was 5 times slower than HNSW on SIFT, which still leaves it far inside the budget. I would not use plain IVF-PQ: with 16 GB there is nothing to compress, and it was the slowest index and needed m = d to reach 0.95. **None of the 10M figures were measured.** My laptop has 7.6 GiB of RAM, so I could not build 10M here; they extrapolate from 1M, and efSearch must be re-tuned on a query sample at the real size.

Q5. Two ways my benchmark could be misleading, and whether I tested them

1. Speeds are noisy and come from one laptop. They are wall-clock times on a WSL2 machine, 3 repeats per point, one session per experiment. *Tested, partly:* the same IVF-PQ settings were timed in Part 2 and again in Part 3 (identical recalls). The median ratio of the two speeds is 0.98, but individual points were up to 36% apart (the Part 2 point behind IVF-PQ's 564 QPS in Table 1 is one of them: Part 3 measured 739 QPS for the same setting), and my two Part 4 runs differed by up to 28%. So speed differences below about 30% should not be trusted, and IVF-PQ's QPS at 0.95 may be understated. The ranking of the indexes at 0.95 (HNSW first by 2.2× or more) is far larger than this noise. *Not tested:* a second machine, and batched or multi-threaded speed for the ANN indexes (only exact search was timed that way).

2. One HNSW build is one sample. hnswlib builds with several threads, so two identical builds give slightly different graphs. *Tested in Part 4 only:* the same rebuilt index reached recall 0.916 in the earlier run and 0.959 in the final one (the earlier run is kept in `results/earlier_run_part4_3repeats.csv`, not in results.csv). I did not repeat the HNSW builds of Part 2, so its HNSW recall and speed carry about the same uncertainty, and I did not search for a better M or efConstruction than the three values tried. *Not tested:* whether the order of the three indexes would change with other build settings.

Also untested: the explanation that GloVe is harder because of its distance contrast, and the profile of why IVF-PQ is slower than IVF-Flat (both stated as likely reasons, not findings).